

En el presente informe se documenta el funcionamiento de los servicios correspondientes a UsuarioService

Se realiza una inyección de dependencia por medio de constructor brindando lo necesario para el momento de su creación. Con esto garantizamos la inmutabilidad de los datos.

Métodos especiales

BuscarUsuarios

```
public List<Usuario> buscarUsuarios(String keyword, String rol) {
    if (rol != null && !rol.isEmpty()) {
        return usuarioRepository.findByRoles_NombreRol(rol); //filtrado por roles
    }
    if (keyword != null && !keyword.isEmpty()) {
        return
usuarioRepository.findByUsuarioContainingIgnoreCaseOrCorreoContainingIgnoreCase(keyword, keyword);
    }
    return usuarioRepository.findAll();
}
```

Este método es usado para cuando el Administrador entre a su panel y seleccione gestión de usuarios pueda acceder a todos los usuarios del sistema incluido el mismo, posibilitando que exista un filtrado por usuario (mportilla46) o correo (dpmon....@gmail.com). Los datos son traídos desde el repositorio donde se le indica que busque ignorando mayúsculas o minúsculas y traiga el dato ya sea el de el usuario o el del correo

```
List<Usuario>
findByUsuarioContainingIgnoreCaseOrCorreoContainingIgnoreCase(String usuario,
String email);
/*se le dice a spring data JPA, que busque en la tabla usuarios donde hay una
relacion con la tabla roles y dentro de esa
* tabla busque el nombre del rol*/
List<Usuario> findByRoles_NombreRol(String nombreRol);
```

se utiliza una lista debido a que puede haber más de un usuario con el correo iniciando en d o su usuario iniciando con m si el administrador hiciera ese filtrado, por ello es necesario pasarle como parámetro la **keyword**, que es la palabra que el administrador usa para filtrar las búsquedas.

Esto se realiza mediante el @RequestParam del controlador UsuarioController

```
@GetMapping
public String listarUsuarios(@RequestParam(name = "keyword", required =
false) String keyword,
                             @RequestParam(name = "rol", required = false)
String rol, Model model) {
    // Si hay keyword, el servicio filtra; si no, trae todos.
    model.addAttribute("usuarios", usuarioService.buscarUsuarios(keyword,
rol));
    model.addAttribute("keyword", keyword); // Para que el texto no se borre
del input al buscar
```

```

model.addAttribute("rolSeleccionado", rol); // para el filtrado de roles
return "admin/usuarios";
}

```

Donde SpringBoot mediante el `@RequestParam` le dice a Java, busque en la URL la keyword y obtenga el dato.

Navegador (HTML): envió este dato bajo el nombre keyword

Protocolo HTTP: Viaja en la URL como `?keyword=valor`.

Servidor (Spring): Tengo un método que espera un parámetro llamado keyword y coinciden, en caso de que no sea así, mostrará toda la lista completa.

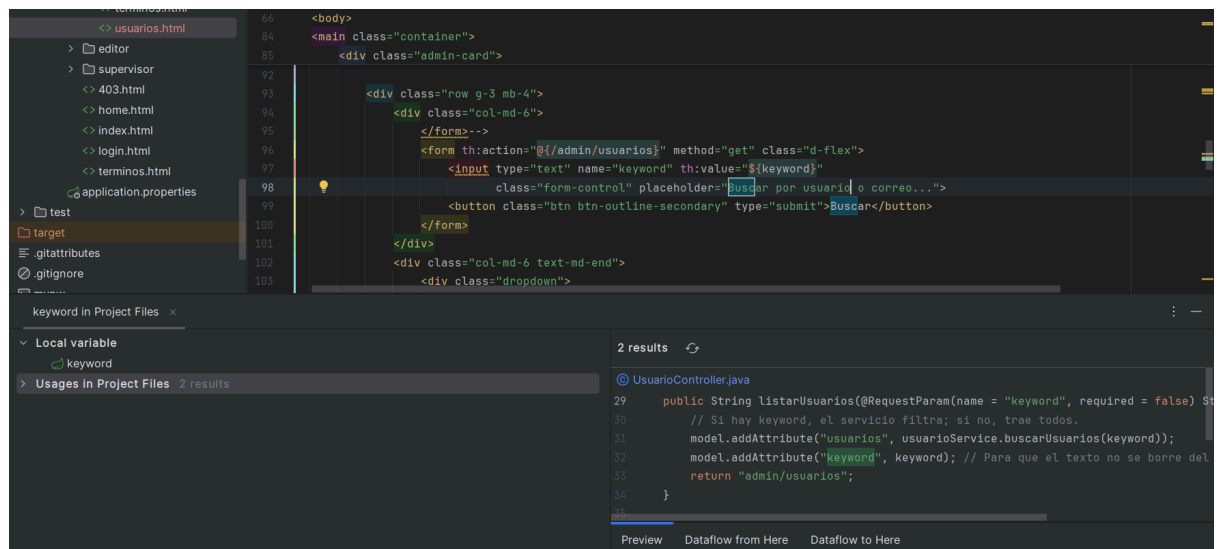
En el HTML se definió de la siguiente manera.

para la búsqueda de usuarios

```

<div class="row g-3 mb-4">
  <div class="col-md-6">
    </form>-->
    <form th:action="@{/admin/usuarios}" method="get" class="d-flex">
      <input type="text" name="keyword" th:value="${keyword}"
        class="form-control" placeholder="Buscar por usuario o
correo...">
      <button class="btn btn-outline-secondary"
type="submit">Buscar</button>
    </form>
  </div>

```



Para el filtrado de roles

Al igual que con la búsqueda de texto, utilizamos `@RequestParam` en el controlador para recibir un String desde la URL. Este parámetro se utiliza en el repositorio para realizar una consulta que navega por la relación de muchos a muchos de roles y busca una coincidencia exacta con el atributo `nombreRol` de la entidad `Rol`. De esta forma, el sistema filtra y devuelve únicamente la lista de usuarios que tienen asignado ese rol específico en la base de datos.

```
<div class="col-md-6 text-md-end">
  <div class="dropdown">
    <button class="btn btn-outline-secondary dropdown-toggle"
type="button" data-bs-toggle="dropdown">
      Filtrar por Rol
    </button>
    <ul class="dropdown-menu">
      <li><a class="dropdown-item"
th:href="@{/admin/usuarios}">Todos</a></li>
      <li><a class="dropdown-item"
th:href="@{/admin/usuarios(rol='ADMIN')}">Administradores</a></li>
      <li><a class="dropdown-item"
th:href="@{/admin/usuarios(rol='SUPERVISOR')}">Supervisores</a></li>
      <li><a class="dropdown-item"
th:href="@{/admin/usuarios(rol='EDITOR')}">Editores</a></li>
    </ul>
  </div>
</div>
```